

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет Информационных технологий и управления
Кафедра Интеллектуальных информационных технологий

РАСЧЕТНАЯ РАБОТА

по дисциплине «Представление и обработка информации в интеллектуальных системах»
на тему

**Найти множество вершин, при удалении которых изменится количество компонент
связности**

Выполнил:

Ю .С. Ануфриев

Студент группы
421701

Проверил:

Н. В. Малиновская

Минск 2025

1 Список понятий

Введение :

Цель: Изучить основы теории графов, способы представления графов, базовые алгоритмы для работы с графами. Получить навыки формализации и обработки информации с использованием семантических сетей.

Задача: Проверить граф на двусвязность.

1. **Граф** - математическая абстракция реальной системы любой природы, объекты которой обладают парными связями. (совокупность точек, соединенных линиями. Точки называются вершинами, или узлами, а линии – ребрами, или дугами.) :

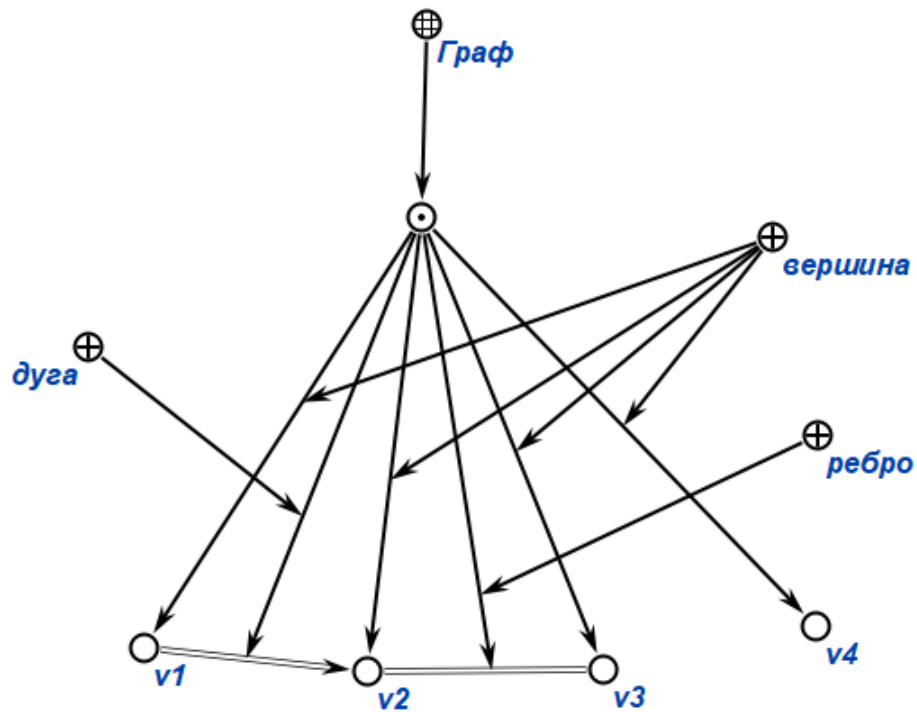


Рис. 1: пример графа

2. **Неориентированный граф** — Граф, ни одному ребру которого не присвоено направление.

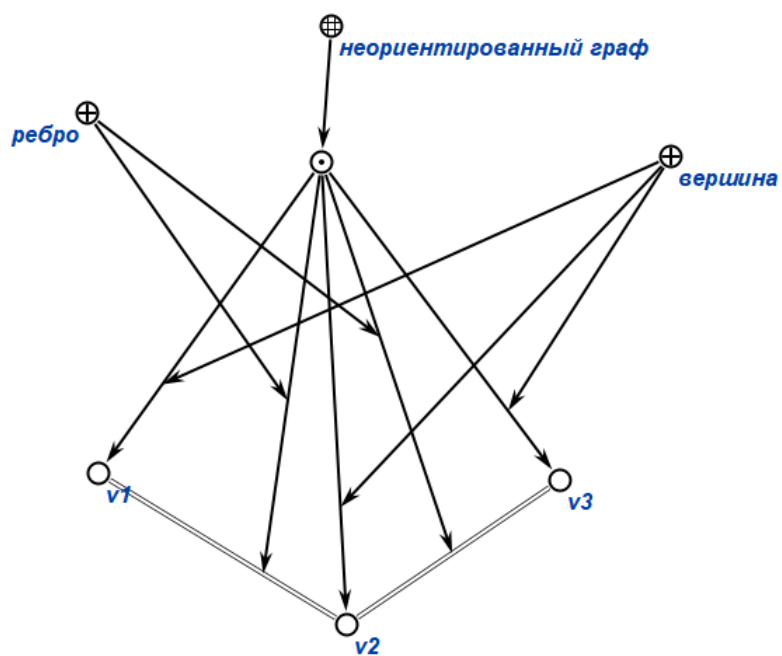


Рис. 2: пример неориентированного графа

3. **Оrientированный граф** — граф, рёбрам которого присвоено направление.

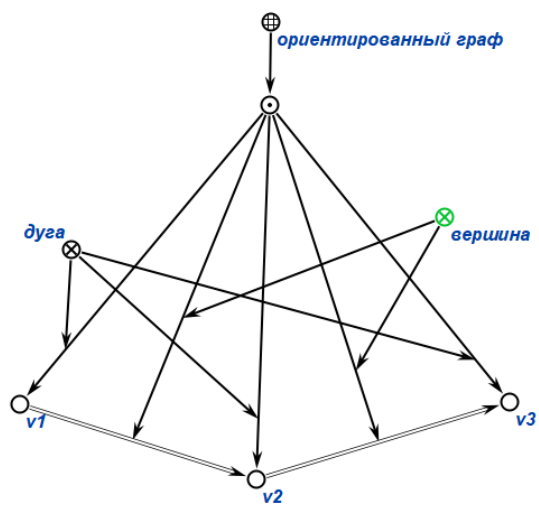


Рис. 3: пример ориентированного графа

4. *Компонента связности* — множество всех связанных вершин графа.

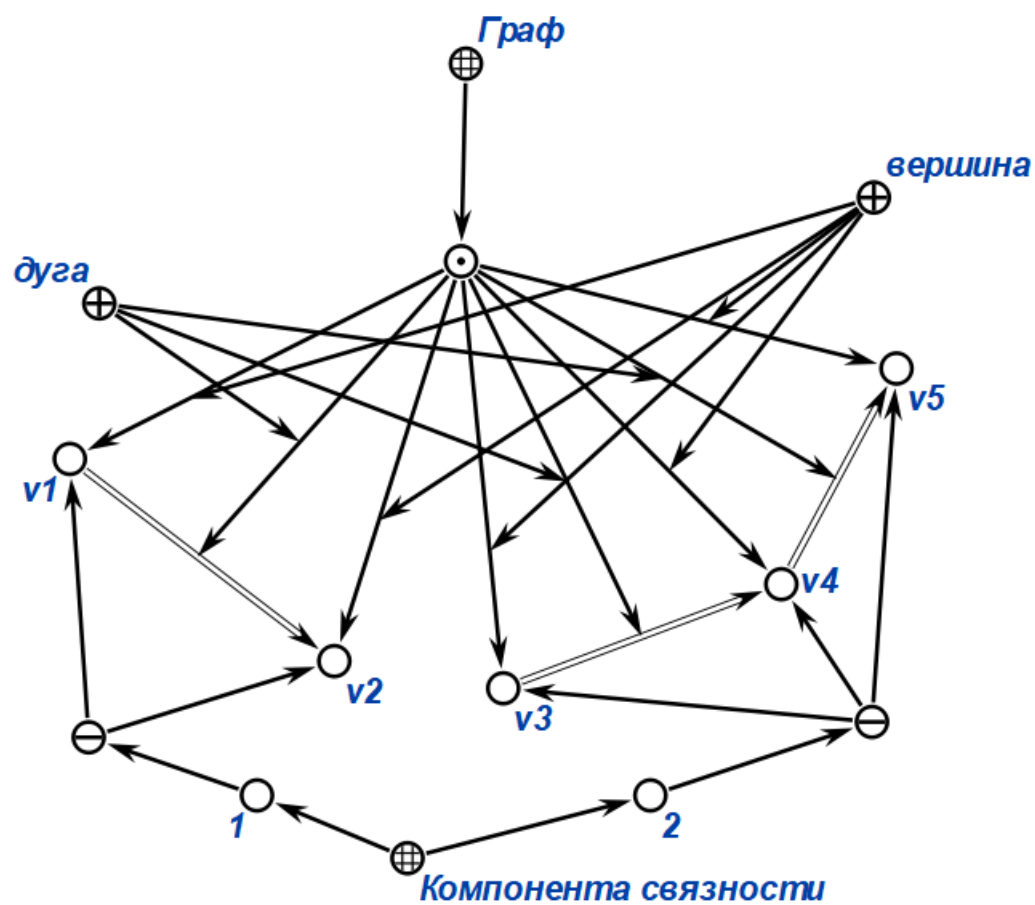


Рис. 4: компоненты связности

2 Примеры графов

2.1 Пример 1

Вход: Необходимо найти множество вершин, удаление которых приводит к увеличению числа компонент связности графа. Переменная n хранит число компонент связности входного графа.

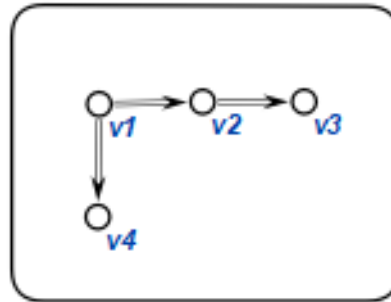


Рис. 5: вход примера 1

Выход: Удаление вершин $v1$ и $v2$ приводит к увеличению числа компонент связности хранящихся в переменной n .

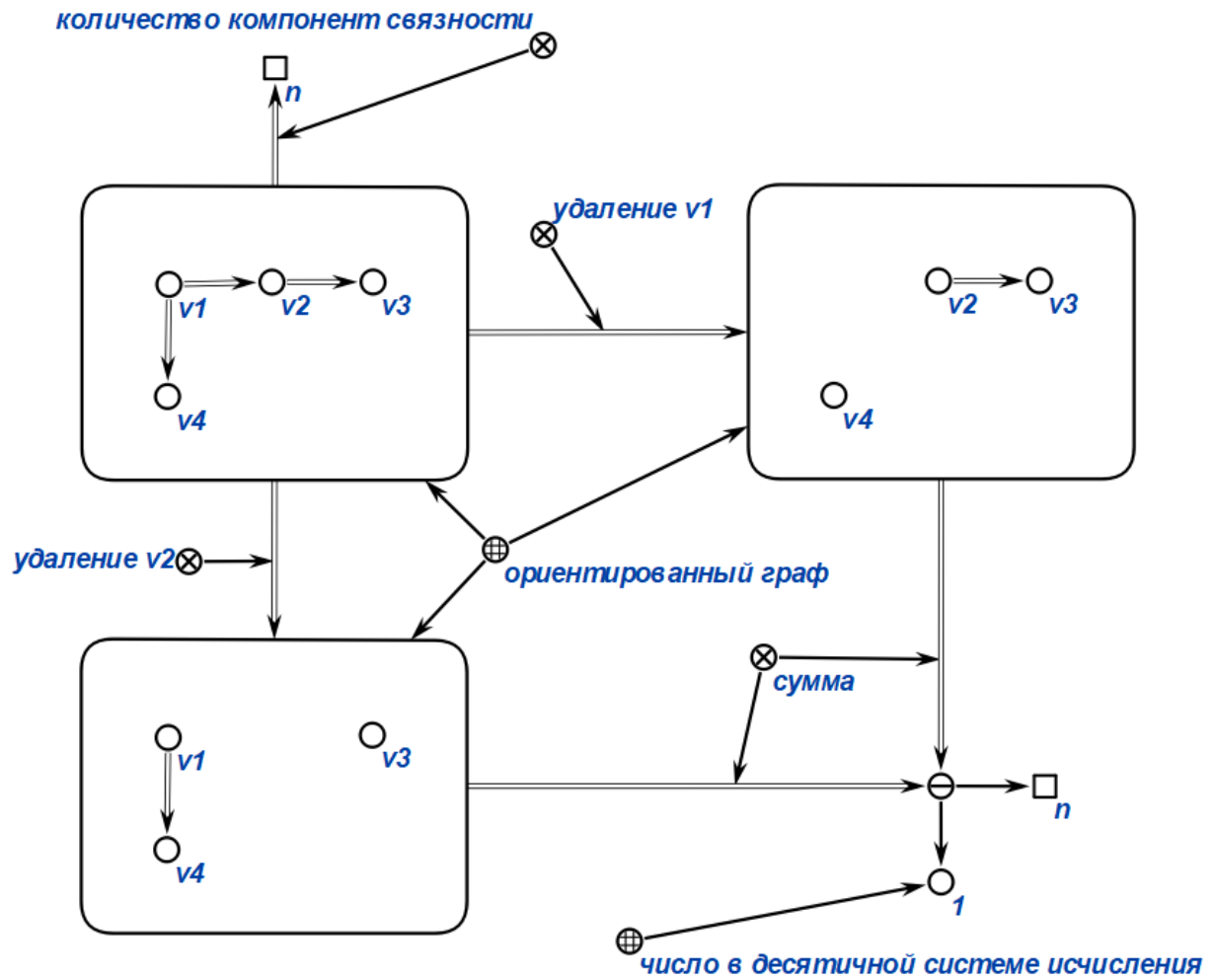


Рис. 6: выход примера 1

2.2 Пример 2

Вход: Найти множество вершин, удаление которых приводит к увеличению числа компонент связности ориентированного графа.

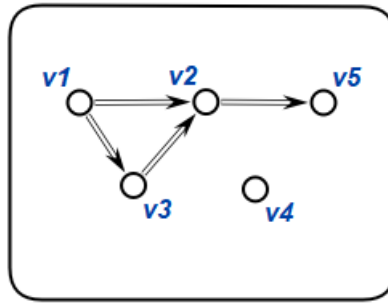


Рис. 7: вход примера 2

Выход: Вершина v2

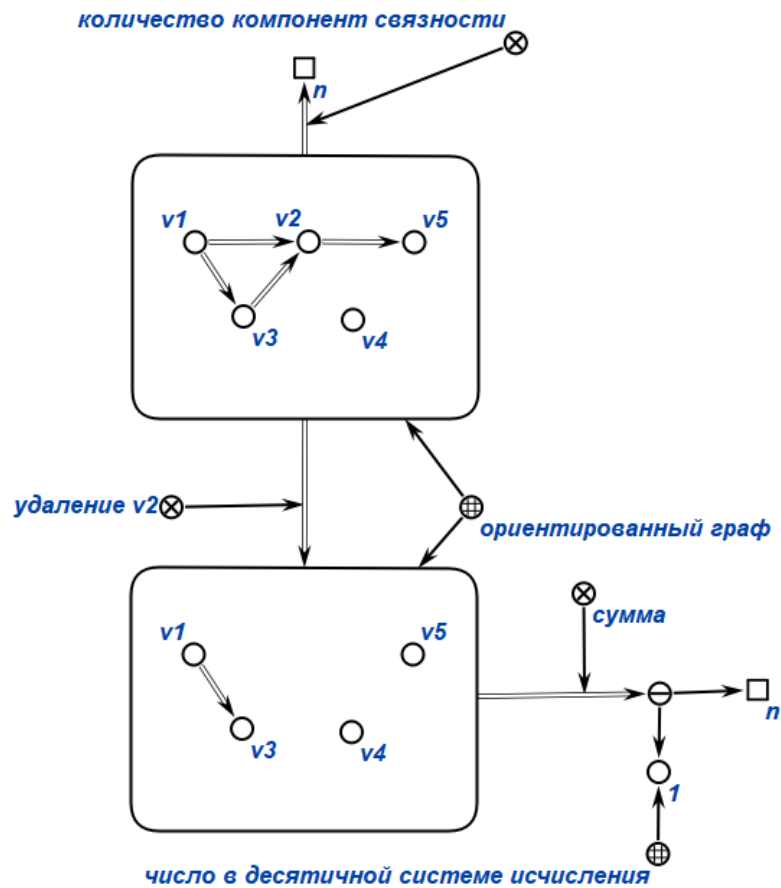


Рис. 8: выход примера 2

2.3 Пример 3

Вход: Найти множество вершин, удаление которых приводит к увеличению числа компонент связности ориентированного графа.

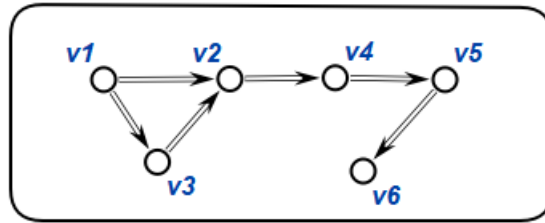


Рис. 9: вход примера 3

Выход: Вершины v2, v4, v5

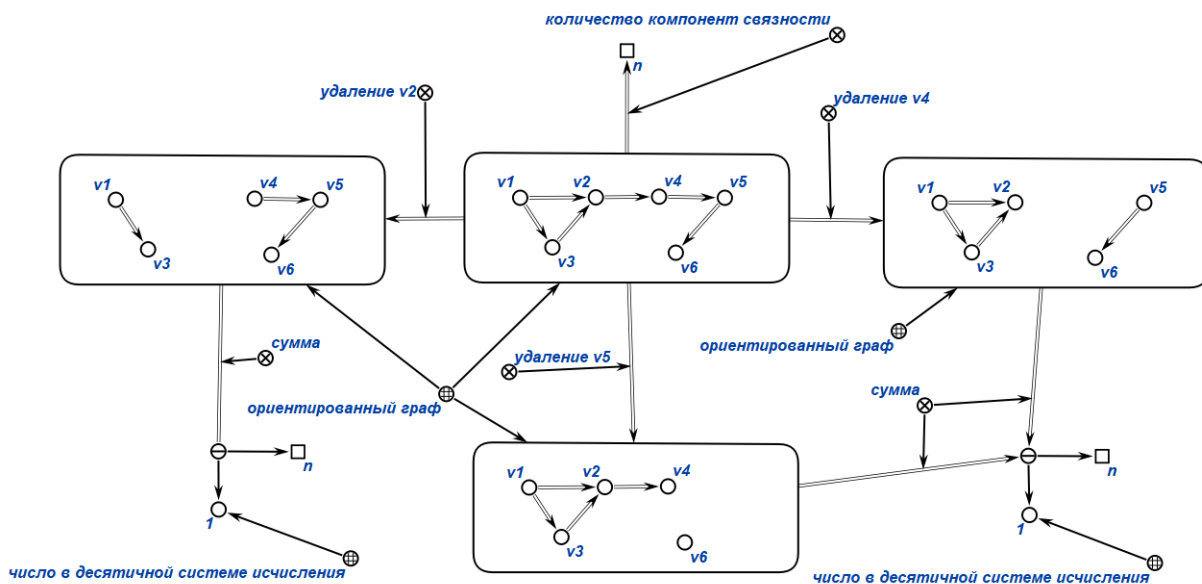


Рис. 10: выход примера 3

2.4 Пример 4

Вход: Найти множество вершин, удаление которых приводит к увеличению числа компонент связности ориентированного графа.

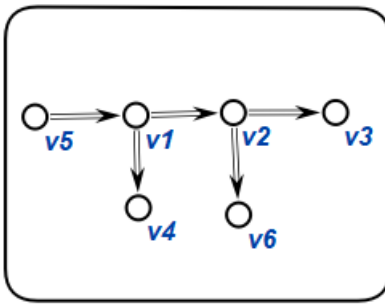


Рис. 11: вход примера 4

Выход: Вершины v1, v2

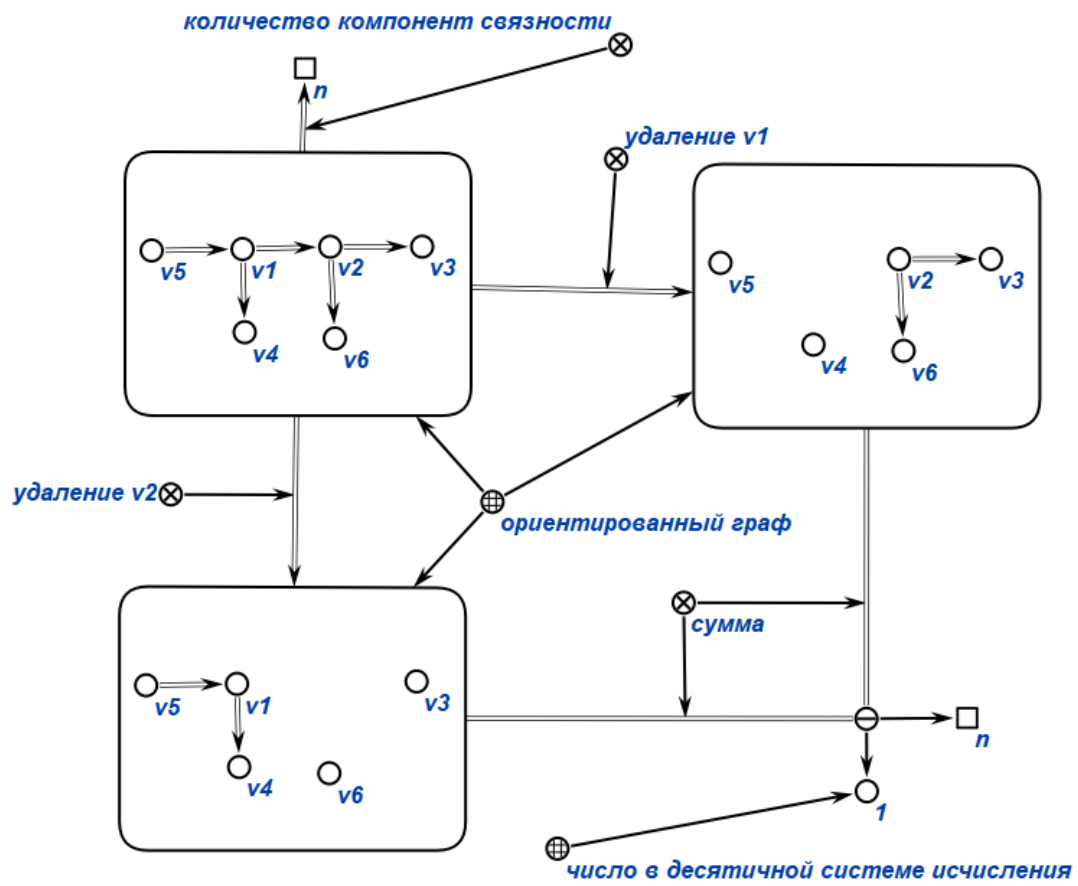


Рис. 12: выход примера 4

2.5 Пример 5

Вход: Найти множество вершин, удаление которых приводит к увеличению числа компонент связности ориентированного графа.

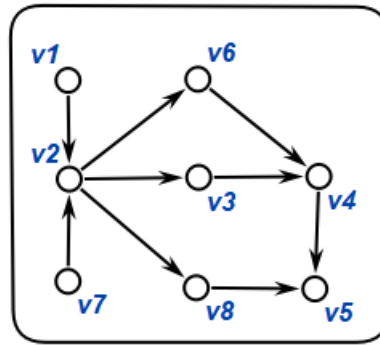


Рис. 13: вход примера 5

Выход: Вершина v2

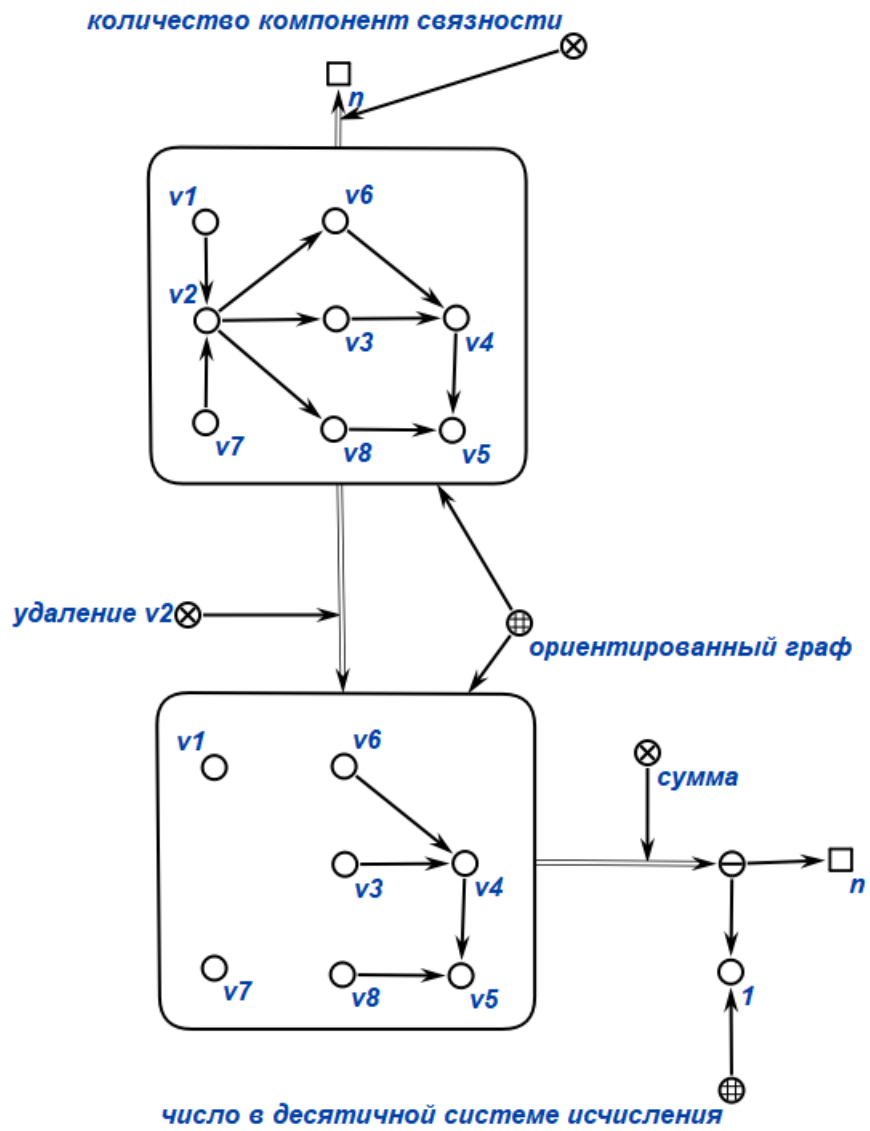


Рис. 14: выход примера 5

3 Алгоритм решения теоретико-графической задачи

1. Получаем в качестве входного графа пользовательский граф. Создаём неориентированный граф из входного;
2. Создаём переменную `originalConectCount` Которая будет хранить количество компонентов связности во входном графе, счетчик `ConectCount` который будет считать количество компонентов связности в каждом графе и `result` хранящий результат;

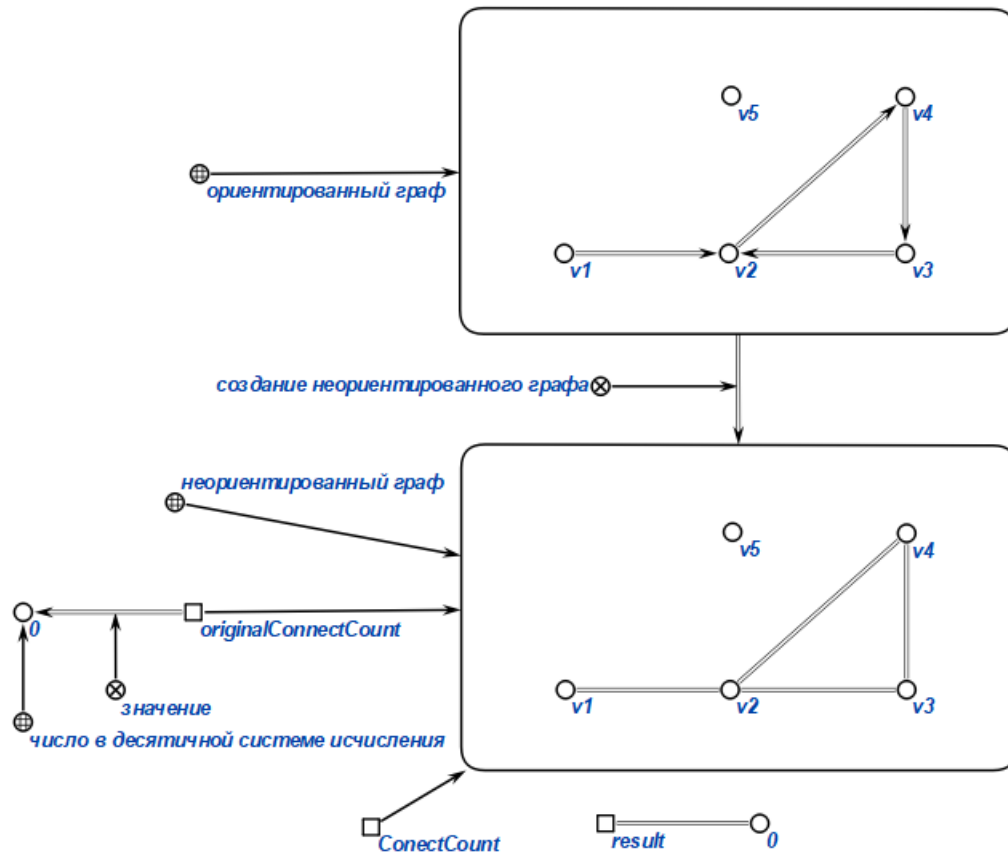


Рис. 15: инициализация переменных

3. Создаём переменную `unvisited` - стек из непосещённых вершин;

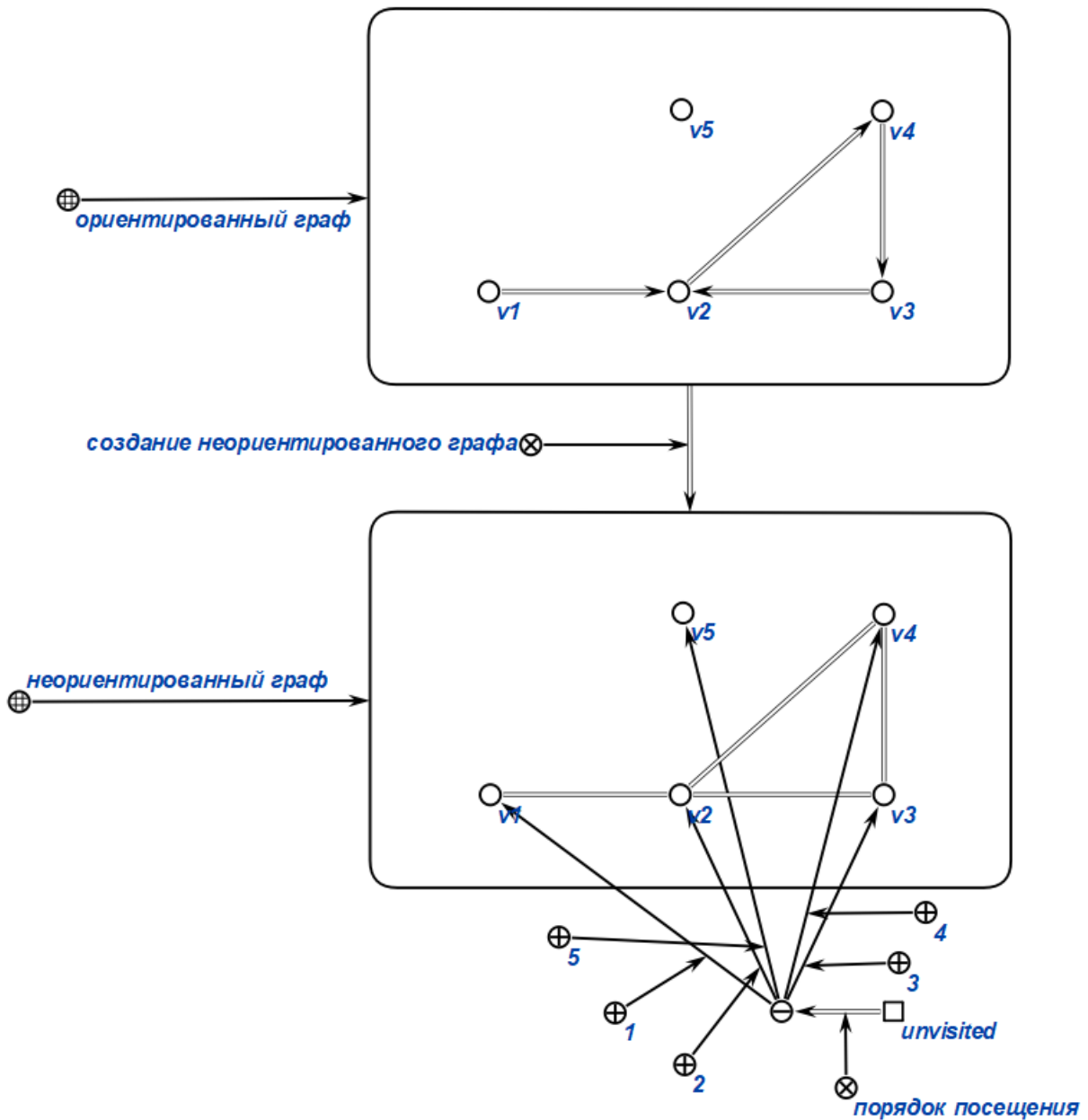


Рис. 16: создание стека непосещенных вершин

4. Создаём стек delete который будет поочереди удалять каждую вершину входного графа;

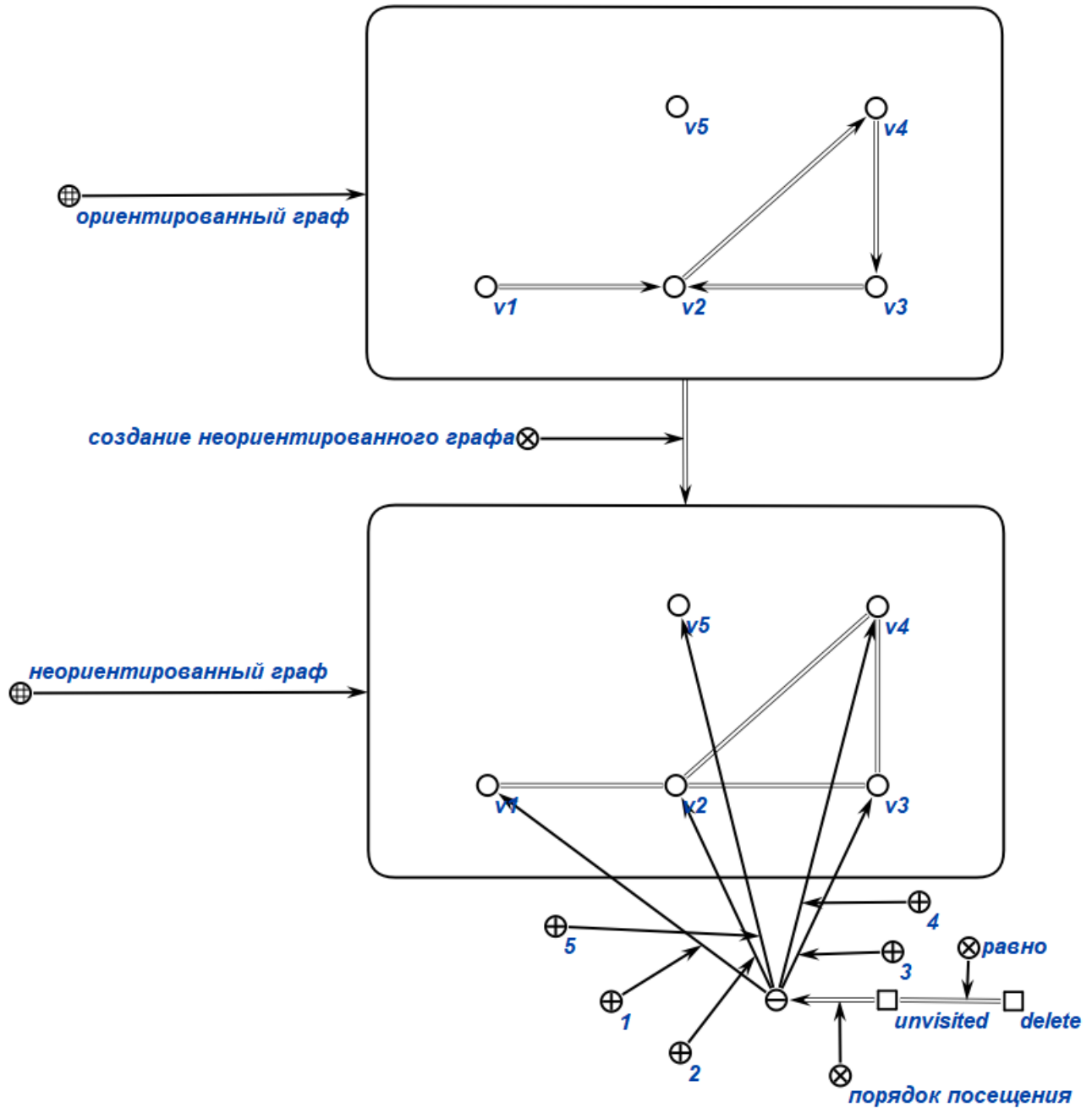


Рис. 17: создание стека вершин для удаления

5. Выбираем и удаляем из стека первую вершину `inputGraph`; `end` показывает последнюю вершину до которой есть связанные с нашей вершиной рёбра;

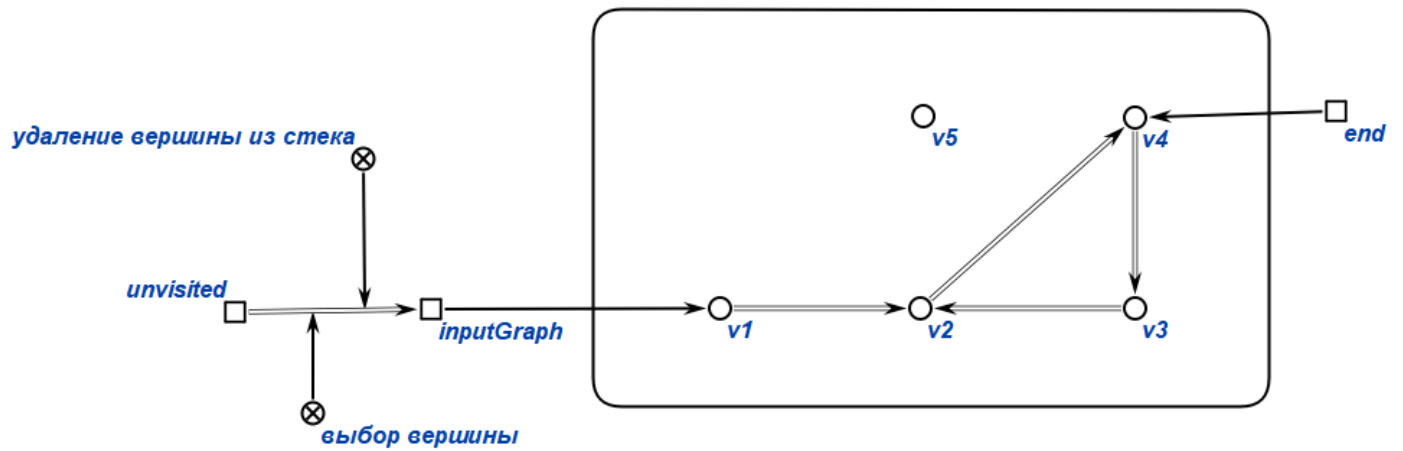


Рис. 18: удаление вершины и обход графа

- После достижения конечной вершины количество компонентов связности ConectCount увеличивается на 1 (изначально в переменной хранится 0); А также каждая посещённая вершина удаляется из стека;

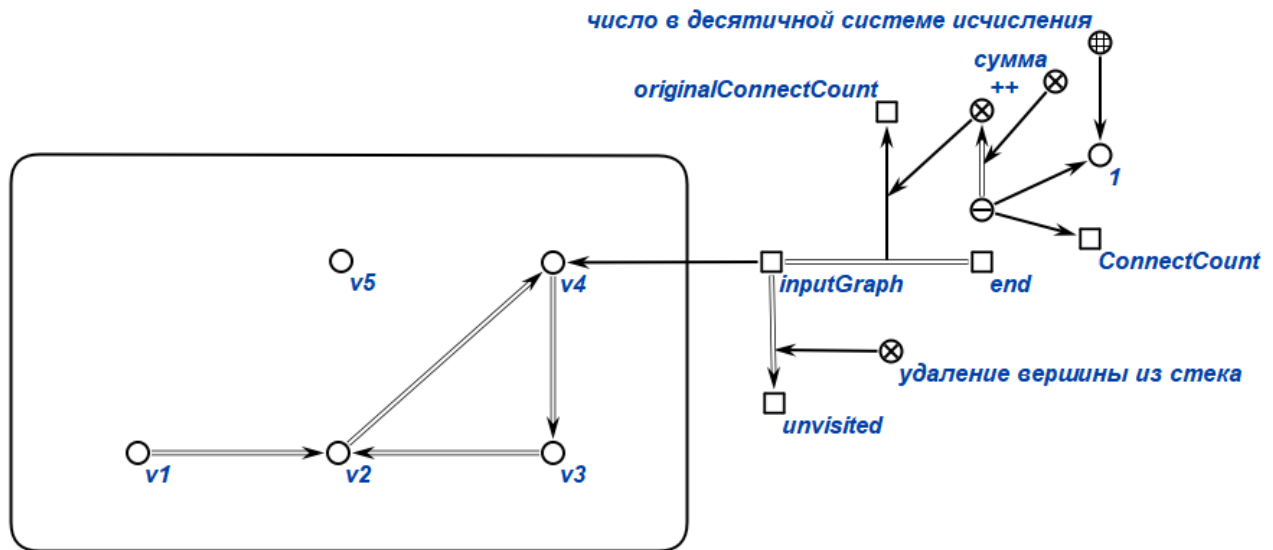


Рис. 19: обновление счетчика ConectCount

- При нахождении отдельно стоящей вершины количество компонентов связности также увеличивается на 1;
- При достижении стеком unvisited нуля количество компонентов связности из ConectCount переходит в переменную originalConectCount. Это означает что теперь входной граф содержит данное количество компонентов связности;

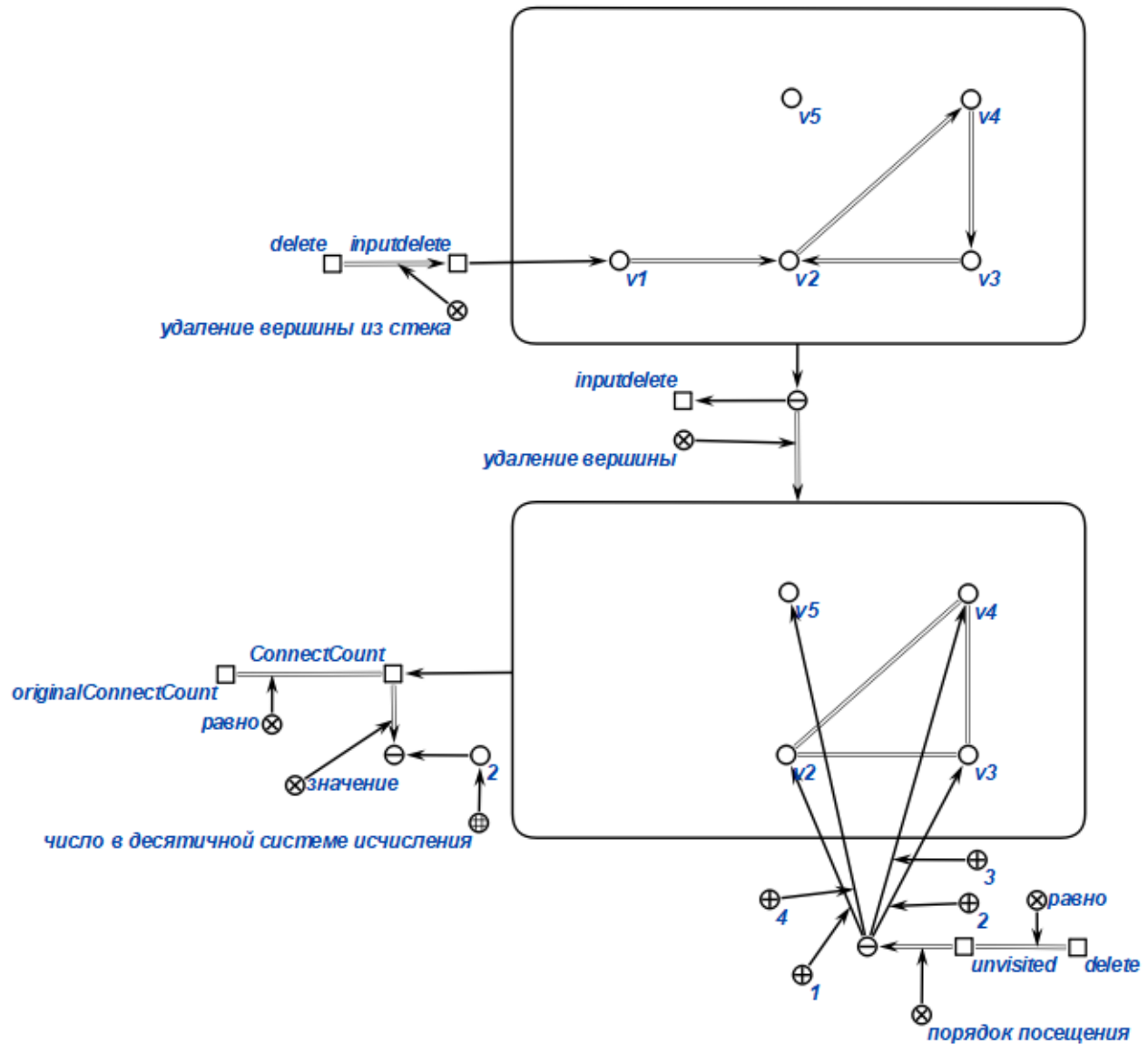


Рис. 22: подсчет компонент связности для графа без одной вершины

12. Повторяем алгоритм пока стек `delete` не опустеет. Если значение в каком-либо графе окажется больше чем во входном - то эта вершина добавляется в `result` для дальнейшего вывода результата;

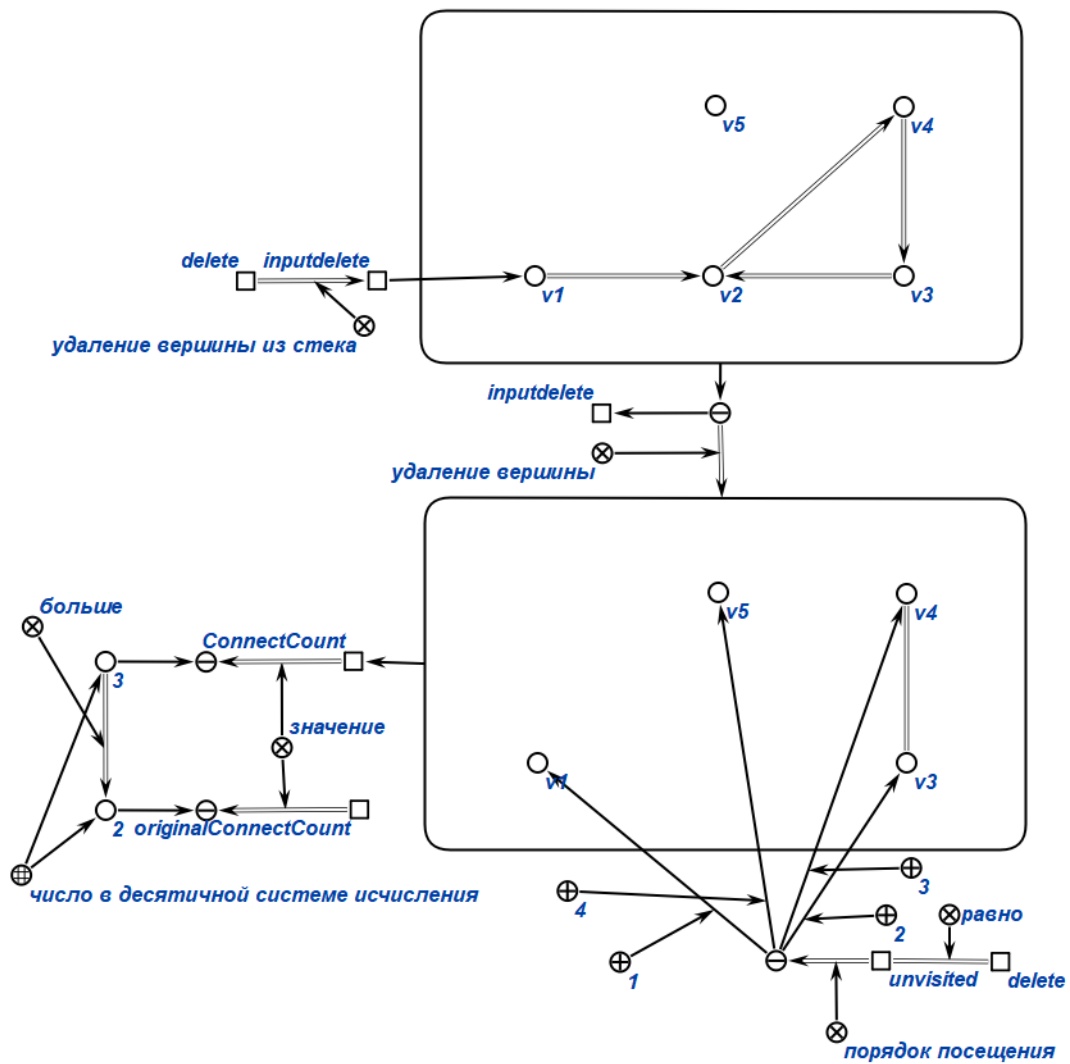


Рис. 23: повторение предыдущего действия для другой вершины

13. Вывод хранящейся в переменной result информации как результат выполнения задачи;

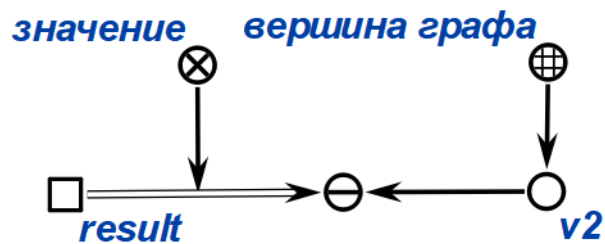


Рис. 24: значение переменной result

4 Выводы

Получил навыки формализации и обработки информации с использованием семантических сетей на примере работы алгоритма конкретной задачи.

5 Литература

- [1] Емеличев В. А., Мельников О. И., Сарванов В. И., Тышкевич Р. И. Лекции по теории графов. М.: Наука, 1990. 384с. (Изд.2, испр. М.: УРСС, 2009. 392 с.)
- [2] Оре О. Теория графов. – 2-е изд.. – М.: Наука, 1980. – С. 336.
- [3] <https://www.youtube.com/watch?v=Z4tAyq8txDg>
- [4] <https://www.youtube.com/watch?v=iQabtd6VBL0>
- [5] <https://www.youtube.com/watch?v=3-XLRh2M5YI>